



KaamMilega™ Flutter Application

Complete Application Analysis — Architecture, UI, Data, State & API

Candidate / Gig-Worker Mobile Client · Full Documentation

Project Path: D:\Desktop\Kaammilega App\KaamMilegaApp

Package: kaam_milega • **Version:** 1.0.0+1 • **Dart SDK:** ^3.13.3

Backend: km-backend (Go / Fiber REST + WebSocket) — <https://api.kaammilega.com/api>

Coverage: 45 screens · 15 models · 12 repositories · 12 providers · 10 shared widgets · 4 test suites

Document Type: Read-only analysis (no source code modified)

Generated: September 22, 2026 · Revision 2 (full-app expansion)

Table of Contents

1. Executive Summary — What This App Is
2. Tech Stack, Dependencies & Platform Configuration
3. Project Architecture & Folder Structure
4. Application Boot, Routing & Navigation Shell
5. Design System — Colors, Typography, Theme & Branding
6. Reusable Widget Library (shared/widgets)
7. Data Models — Complete Data Dictionary
8. State Management — Full Provider Catalog
9. **API Layer — In-Depth Breakdown**
 - 9.1 ApiClient (Dio) & interceptors
 - 9.2 Error translation & AppException hierarchy
 - 9.3 ApiConstants & endpoint catalog
 - 9.4 All 12 repositories explained
 - 9.5 WebSocket real-time chat
 - 9.6 Session lifecycle & endpoint status matrix
 - 9.7 End-to-end request traces
10. Feature-by-Feature Screen Walkthrough (all 45 screens)
11. Offline Resilience & Caching System
12. Testing, Git History & Existing Documentation
13. Observations, Risks & Recommendations

1. Executive Summary — What This App Is

KaamMilega ("Kaam Bhi. Skill Bhi. Kamaai Bhi.") is a Flutter mobile application serving **candidates and gig workers in India**. It is a **thin, API-driven client**: virtually no business logic lives locally — screens call a remote Go backend (`km-backend` , Fiber framework) over REST, plus one persistent WebSocket for real-time chat.

Product domains implemented in the app:

Domain	What the user can do	Backing API / notes
Auth	Phone-OTP login, email+password login & register, forgot/reset password (2-step), email OTP verification	<code>/auth/*</code> → JWT
Profile / CV	Full CV screen: headline, about, education, experience, projects, skills, portfolio, profile & cover photo upload/delete, resume upload, profile-strength meter	<code>/user/*</code> + <code>/files/upload</code>
Jobs	Search (400 ms debounce), city/type/salary/experience/gender/qualification filters, pagination, save/bookmark, job detail, 1-click apply with cover letter, call HR, chat with HR	<code>/jobs</code> , <code>/applications</code>
Career tracking	My Applications (status pills), Application detail, Interview schedule	<code>/applications/my</code> , <code>/interviews/my</code>
Chat	Conversation list, message history, live duplex messaging with connection status	REST <code>/chats</code> + WS <code>/ws/chats</code>
Network / P2P	Send/accept/ignore/delete connections, pending invitations, peer discovery	<code>/network/*</code>
Wallet	Balance card, wallet-type breakdown, KYC & limits card, transaction ledger with credit/debit filter, add money, withdraw, P2P transfer	<code>/wallet/*</code> (backend routes partially pending)
Discover modules	Explore hub → Instant Work, Skills Marketplace, Experts/Mentorship, Services, Peer-to-Peer, Events, Feed/Resources, Company page	<code>/events</code> , <code>/mentorships</code> , <code>/skills</code> , <code>/cities</code> , <code>/posts</code>
System	Global offline banner, retry views, guest browsing with auth prompts, notifications, settings, city selector	<code>/notifications</code>

Design philosophy (from `API_INTEGRATION_STATUS.md`)

1) **No fake/mock production data** — unimplemented backend routes must never be disguised as empty datasets. 2) **Errors classified distinctly** — 404 = "service under development", 401/403 = session expired, 5xx = server issue, network failure = offline banner + cached data + auto-retry, `200 []` = genuinely empty. 3) **Single-line structured logs** instead of raw Dio stack traces.

Scale snapshot: 45 presentation files (largest: `profile_screen.dart` 6,781 lines, `home_screen.dart` 1,588, `job_detail_screen.dart` 1,471, `wallet_screen.dart` 1,020, `jobs_screen.dart` 920) · 15 model classes · 12 repositories · ~25 providers · 30+ routes · 72 tracked product features (35 fully implemented, 17 partial, 5 pending, 15 upcoming).

2. Tech Stack, Dependencies & Platform Configuration

Package	Version	Role
<code>flutter_riverpod</code>	<code>^3.4.3</code>	State management + DI. All repositories/services are Providers; screens consume reactive state.
<code>go_router</code>	<code>^18.0.1</code>	Declarative routing: paths, path params, <code>extra</code> objects, redirect.
<code>dio</code>	<code>^5.11.1</code>	HTTP client — wrapped entirely by the app's own <code>ApiClient</code> .
<code>shared_preferences</code>	<code>^2.5.5</code>	JWT, user JSON, city, saved jobs, offline caches.
<code>connectivity_plus</code>	<code>^7.3.1</code>	OS online/offline events → global network status.
<code>cached_network_image</code>	<code>^4.0.0</code>	Disk-cached avatars/banners.
<code>shimmer</code>	<code>^3.0.0</code>	Skeleton loaders while data is in flight.
<code>intl</code>	<code>^0.20.3</code>	INR number formatting (<code>#,##,###</code> en_IN), dates/times.
<code>url_launcher</code>	<code>^6.3.2</code>	Open links, tel: calls to HR, resume viewing.
<code>image_picker</code> / <code>file_picker</code>	<code>^1.1.2</code> / <code>^8.1.4</code>	Profile/cover photos · PDF/DOC resume selection.
<code>cupertino_icons</code>	<code>^1.0.8</code>	iOS-style icons.
<code>flutter_lints</code> (dev)	<code>^6.0.0</code>	Lint ruleset.

Platform configuration

- **Platforms scaffolded:** Android, iOS, Web, Windows, Linux, macOS.
- **Android applicationId:** `com.example.kaam_milega` — still the Flutter default `com.example` namespace (must be changed before Play Store release).
- **Java/Kotlin:** JVM 17; Gradle uses the Flutter Gradle Plugin.
- **Release signing:** currently uses **debug keys** (TODO in `build.gradle.kts`).
- **Android permissions:** `INTERNET` , `ACCESS_NETWORK_STATE` only — no camera/location/notifications yet (matches pending features: GPS tracking, FCM push, image capture via plugin-manifest merges).
- **App label:** "Kaam Milega"; `singleTop` launch mode; predictive back enabled.
- **analysis_options:** includes `flutter_lints/flutter.yaml` , excludes all platform/build folders.

3. Project Architecture & Folder Structure

Feature-first, Clean-Architecture-lite: infrastructure in `core/` + `shared/`; each business feature owns `models` → `repositories` → `providers` → `presentation`.

```
KaamMilegaApp/
├── lib/
│   ├── main.dart                // entry: LocalStorage.init → ProviderScope → app
│   ├── app/
│   │   ├── app.dart             // MaterialApp.router + global OfflineBannerOverlay
│   │   ├── router.dart          // ~30 GoRouter routes
│   │   └── theme/ app_theme.dart · app_colors.dart · app_text_styles.dart
│   ├── core/
│   │   ├── constants/api_constants.dart    // base URL + every endpoint path + resolveImageUrl
│   │   ├── constants/app_branding.dart     // brand names, taglines, trust signals
│   │   ├── network/
│   │   │   ├── api_client.dart            // ★ Dio wrapper: interceptors + error mapping
│   │   │   ├── app_exception.dart         // ★ sealed AppException hierarchy
│   │   │   ├── connectivity_service.dart   // singleton status + DNS probe
│   │   │   ├── connectivity_provider.dart  // networkStatusProvider / isOnlineProvider
│   │   │   └── network_status.dart · offline_banner.dart
│   │   └── storage/local_storage.dart      // SharedPreferences: token, user, 3 caches
│   ├── shared/widgets/ 10 reusable widgets (see §6)
│   └── features/<feature>/{models, repositories, providers, presentation}
├── test/ 4 test suites (see §12)
├── android/ ios/ web/ windows/ linux/ macos/
├── API_INTEGRATION_STATUS.md · flutter_app_feature_tracker.{md,csv}
├── KAAMMILEGA_FLUTTER_PROJECT_DOCUMENTATION.{md,pdf}
└── pubspec.yaml · analysis_options.yaml · README.md
```

Dependency direction (enforced everywhere)

Screen (presentation) → Riverpod Notifier (providers) → Repository → ApiClient (Dio) → <https://api.kaammilega.com/api>

Screens never touch Dio; repositories never build UI; only `ApiClient` knows HTTP details.

4. Application Boot, Routing & Navigation Shell

```
main() → ensureInitialized → LocalStorage.init() → runApp(ProviderScope(KaamMilegaApp))
KaamMilegaApp → MaterialApp.router(theme, routerConfig) + builder wraps all screens in OfflineBannerOverlay
SplashScreen (1.5 s): logo + tagline + spinner → context.go('/home') // guests allowed (same destination)
AuthNotifier.build() (parallel): token? → hydrate cached user → GET /user/profile
```

Full route table (GoRouter, initialLocation: /splash)

Path	Name	Screen / params
/splash	splash	SplashScreen
/	—	redirect → /home
/home · /jobs · /chats · /profile	home/jobs/chats/profile	MainNavigationShell with initialIndex 0 / 1 / 3 / 4
/login · /register · /otp · /forgot-password	auth stack	LoginScreen · RegisterScreen · OtpScreen(phone via <code>extra</code>) · ForgotPasswordScreen(email via <code>extra</code>)
/jobs/:id	job_detail	JobDetailScreen(jobId, optional Job object via <code>extra</code> for instant paint)
/saved-jobs · /my-applications · /applications/:id · /interviews	career	SavedJobs · MyApplications · ApplicationDetail · Interviews
/chats/:id	chat_detail	ChatDetailScreen — <code>extra</code> map: {receiverId, title}
/network · /notifications · /events · /company/:id	social	NetworkScreen · NotificationsScreen · EventsScreen · CompanyScreen(companyId)
/wallet + /wallet/{transactions,add-money,withdraw,transfer}	wallet	5 wallet screens
/explore · /instant-work · /skills-marketplace · /experts · /services · /peer-to-peer · /feed	modules	Explore hub + 6 module screens + Resources/Feed screen
/apply-expert · /settings	misc	ApplyExpertScreen · SettingsScreen

Bottom navigation shell (5 tabs + center action)

Idx	Tab	Screen	Behaviour
0	Home	HomeScreen	All tabs live in an IndexedStack (state preserved when switching). Custom bottom bar (white, shadow, SafeArea) — no default MaterialNavigationBar. Chats tab has a red unread badge (currently hard-wired to 0 pending backend unread counts). Each screen receives an <code>onNavigateTab</code> callback to switch tabs programmatically.
1	Jobs	JobsScreen	
2	Center (+)	<i>no screen</i> (SizedBox.shrink)	
3	Chats	ChatListScreen	
4	Profile	ProfileScreen	

56 px gradient circle (Brand Blue → Deep Navy, glow shadow) opens a **Quick Actions** bottom sheet: Instant Work Request (snackbar only — not yet wired to API), Browse Resources → /feed, Browse Live Events → /events

5. Design System — Colors, Typography, Theme & Branding

5.1 Brand colors (AppColors — single source of truth)

Token	Hex	Usage
brandBlue / primary	#1A2B8C	Primary brand blue — all primary interactive elements, active tab, focus borders
brandOrange / accent	#F97316	Accent CTAs, InstantMilega, highlights
deepNavy / primaryDark	#0D1B5E	Dark headers, hero background, gradient end, wallet balance card
primaryLight	#EFF6FF	Blue-50 backgrounds
accentLight / accentDark	#FFF7ED / #EA580C	Orange-50 bg / hover
background / surface	#F8FAFC / #FFFFFF	Page background / cards
textPrimary/Secondary/Light	#0F172A / #64748B / #94A3B8	Body / subtext / placeholders
border / borderLight	#E2E8F0 / #F1F5F9	Card borders / dividers
success / warning / error	#16A34A / #EAB308 / #DC2626	Status semantics

Per-module colour coding: Jobs #1A2B8C (blue) · InstantWork #F97316 (orange) · Skills #16A34A (green) · Experts #7C3AED (purple) · Services #EC4899 (pink) · P2P #0D9488 (teal) · Events #EA580C (red-orange)

Top Match badge set: gold #FBBF24 on #FFFDF2 with #FDE68A border · **verified pill:** #16A34A on #DCFCE7

5.2 Typography (AppTextStyles — Design System v1.0)

Style	Size / Weight	Notes
displayXl / displayLg	48 w900 (ls -1.0) · 36 w800	Hero numbers/titles
headingXl / headingLg / headingMd	28 w800 · 22 w800 · 18 w700	Aliases: heading1/2/3
bodyLg / bodyMd / bodySm	16 · 14 · 13 (w500, height 1.4)	Aliases: body / bodySecondary
label / micro / button	12 w700 · 10 w600 · 15 w700 white	caption = label

5.3 ThemeData (AppTheme.lightTheme)

- **useMaterial3: true**, scaffold background #F8FAFC, ColorScheme seeded from brand blue.
- AppBar: white, no elevation, left-aligned titles.
- Inputs: filled white, 12 px rounded outline borders, focus border brand blue 1.5 px, 16 px padding.
- ElevatedButton: brand-blue, white text, min 64×48, radius 12, no elevation, w700 15 px. OutlinedButton mirrors it.
- Card: white, radius 16, 1 px border, no elevation/margin. Divider: #E2E8F0 1 px.
- Text theme mapped from AppTextStyles into Material slots.

5.4 Branding (AppBranding)

Brand name **KaamMilega™** (wordmark split "Kaammi"+accent "lega") · taglines "Kaam Bhi. Skill Bhi. Kamaai Bhi." / "Kaam Har Koi, Milega Yahi!" · hero "Kaam Dhoondo. Milega Yahi!" · sub-brand **InstantMilega™** "Get Work in Minutes..." with "ONE TIME ACCESS ₹99 ONLY" · trust signals: Verified & Trusted, AI Powered Matching, Secure Payments, 24×7 Support, One Time Access (₹99), Earn Rewards.

6. Reusable Widget Library (`lib/shared/widgets`)

Widget	Purpose	Key props
<code>NetworkStateView</code>	The standard 5-state wrapper: Loading / Offline / Error+Retry / Empty / Cached-data (amber timestamp badge above content)	<code>isLoading</code> , <code>isOffline</code> , <code>errorMessage</code> , <code>isEmpty</code> , <code>child</code> , <code>onRetry</code> , <code>emptyTitle/Message/Action</code> , <code>cachedTimestamp</code>
<code>CachedDataBadge</code>	"Showing saved data (Updated: ...)" amber strip	<code>timestamp</code>
<code>CategoryTopHeader</code>	Module header: logo→home, search bar, notification bell w/ badge, hamburger drawer, location dropdown	<code>themeColor</code> , <code>searchHint</code> , <code>searchController</code> , <code>scaffoldKey</code>
<code>ThemedCategoryBottomNav</code>	Bottom nav whose colours adapt to the active category theme	<code>activeColor</code> , <code>secondaryColor</code> , <code>activeIndex</code> , <code>categoryLabel/Icon</code>
<code>AppShimmer</code> / <code>ShimmerBox</code> / <code>ShimmerCircle</code> / <code>ShimmerLoadingList</code>	Generic shimmer wrappers & skeleton primitives	<code>width/height/radius</code> , <code>count</code> , <code>itemHeight</code>
<code>JobCardSkeleton</code> / <code>ApplicationSkeleton</code>	Domain-specific skeleton lists (jobs, applications)	—
<code>AuthPromptDialog</code>	Standard "Sign In Required" dialog for guest users	—
<code>AppButton</code>	Consistent button with loading spinner state	<code>text</code> , <code>isLoading</code> , <code>height</code> , <code>onPressed</code>
<code>AppTextField</code>	Styled text field	<code>obscureText</code> , <code>onChanged</code> , <code>enabled</code>
<code>AppLogo</code> / <code>AppBrandBarLogo</code>	Icon-only logo / icon+wordmark lockup	<code>size</code> , <code>showText</code> · <code>iconHeight</code> , <code>textHeight</code> , <code>spacing</code>

7. Data Models — Complete Data Dictionary

All models map the km-backend MongoDB schema, tolerate both `id` and `_id`, defensively default every field, and expose formatted getters using `intl`.

7.1 UserProfile (auth) — the largest model

Fields: `id`, `mobile`, `roles[]`, `isRegistered`, `name`, `email`, `gender`, `city`, `state`, `headline`, `about`, `educationLevel`, `workExperience`, `jobCategories[]`, `experienceDetail`, `skills[]`, **`education[]` (`EducationItem`)**, **`experience[]` (`ExperienceItem`)**, **`projects[]` (`ProjectItem`)**, `profileImage`, `coverImage`, `isEmailVerified`, `resumeUrl`, `openToWork`, `providingServices`, `isAvailableForGigs` (default `true`), `walletBalance`, `profileViewsCount`, `postImpressionsCount`, `searchAppearancesCount`, `portfolioUrl`, `portfolioText`.

Notable: `fromJson` unwraps `{user:{...}}` or `{data:{...}}` envelopes automatically; computed getters `publicProfileUrl` = `www.kaammilega.com/in/{id}` and `fullPublicProfileUrl` (https). This object is what gets cached in `km_user_data`.

7.2 Jobs module

Model	Fields
Job	<code>id</code> , <code>recruiterId</code> , <code>title</code> , <code>description</code> , <code>company</code> , <code>cityId</code> , <code>cityName</code> , <code>location</code> , <code>salaryMin/Max</code> , <code>jobType</code> , <code>status</code> , <code>requirements[]</code> , <code>weOffer[]</code> , <code>gender</code> , <code>education</code> , <code>experienceMin/Max</code> , <code>vacancies</code> (def 1), <code>applicantCount</code> , <code>createdAt</code> . Getters: <code>formattedSalary</code> (₹20,000 - ₹30,000 / "Disclosed on interview"), <code>formattedExperience</code> ("1-3 Yrs"/"Fresher"), <code>formattedLocation</code> ("Andheri, Mumbai"). Defaults: title "Job Opportunity", company "KaamMilega Partner", city "All India".
JobsResponse	<code>jobs[]</code> , <code>total</code> , <code>page</code> , <code>limit</code> · getters <code>totalPages</code> / <code>hasNextPage</code> / <code>hasPreviousPage</code>
JobFilter	<code>searchQuery</code> , <code>city</code> ('All'), <code>jobTypes[]</code> , <code>salaryRange</code> , <code>experience</code> , <code>genders[]</code> , <code>qualification[]</code> , <code>page</code> , <code>limit</code> (10). Helpers: <code>activeFilterCount</code> (badge), <code>hasActiveFilters</code> , and <code>toQueryParams()</code> → <code>page,limit,search,city_ids,job_types,salary_min,experience_max,genders,education</code>

7.3 Career, chat & social models

Model	Fields & behaviour
ApplicationItem	<code>id</code> , <code>jobId</code> , <code>recruiterId</code> , <code>candidateId</code> , <code>status</code> (def "Applied"), <code>coverLetter</code> , <code>createdAt</code> , <code>jobTitle</code> , <code>companyName</code> , <code>cityName</code> , <code>salaryMin/Max</code> , <code>jobType</code> — flattens the nested <code>job:{}</code> object; <code>formattedSalary</code> getter
InterviewItem	<code>id</code> , <code>applicationId</code> , <code>recruiterId</code> , <code>candidateId</code> , <code>scheduledAt</code> , <code>type</code> (def "Phone"), <code>location</code> , <code>status</code> (def "Scheduled"), <code>notes</code> , <code>jobTitle</code> , <code>companyName</code> , <code>recruiterName</code> , <code>recruiterEmail</code> , <code>createdAt</code> — flattens <code>job:{} + recruiter:{} ; formattedScheduleDate</code> = "Tue, 22 Sep 2026 • 3:00 PM"
ChatMessage	<code>id</code> , <code>conversationId</code> , <code>senderId</code> , <code>content</code> , <code>isRead</code> , <code>createdAt</code> · <code>formattedTime</code> "h:mm a"
ConversationItem	<code>id</code> , <code>participants[]</code> , <code>lastMessageId</code> , <code>lastMessage</code> , <code>updatedAt</code> , <code>createdAt</code> · <code>formattedTime</code> (time today, "MMM d" otherwise) · <code>getOtherParticipant(me)</code> for 1-on-1 tiles
ConnectionRequestItem	<code>id</code> , <code>senderId</code> , <code>receiverId</code> , <code>status</code> (def "pending"), <code>createdAt</code> , <code>updatedAt</code>
NotificationItem	<code>id</code> , <code>type</code> (connection/profile_view/job_alert/application/reaction), <code>title</code> , <code>message</code> , <code>avatarUrl</code> (run through <code>resolveImageUrl</code>), <code>isRead</code> , <code>createdAt</code> , <code>targetRoute</code> (deep-link used when tapping a notification) · has <code>toJson</code>

7.4 Marketplace & wallet models

Model	Fields & behaviour
EventItem	id, title, organizer, description, date, time, dateString, location, imageUrl, participants[], attendeesCount, isRegistered, createdAt · <code>copyWith</code> supports optimistic registration; <code>fromJson(json, currentUserId:)</code> derives <code>isRegistered</code> from participants list
ExpertItem	id, expertId, title, description, category, duration, price, rating, reviews, status, expertName, expertHeadline, expertImage, expertBio · parses two shapes : nested <code>{mentorship:{}, expert:{}}</code> or flat JSON (verified by unit test)
SkillItem	id, name, category (def "General") · toJson
City	id, name · toJson + value equality on id
WalletTransaction	id, title, description, amount (double), enum TransactionType (credit/debit), enum TransactionStatus (completed/pending/failed — "success"→completed), createdAt, referenceId?, paymentMethod?, category? · full toJson

8. State Management — Full Provider Catalog

Three Riverpod patterns are used consistently:

Pattern	Used by	Purpose
<code>Provider<T></code>	<code>apiClientProvider</code> , 12 × repository providers, <code>connectivityServiceProvider</code> , <code>chatWebSocketServiceProvider</code> , <code>unreadNotificationsCountProvider</code> , <code>isOnlineProvider</code> , <code>connectionStatusProvider</code> (family)	Dependency injection
<code>Notifier<State></code>	<code>authProvider</code> , <code>jobsProvider</code> , <code>walletProvider</code> , <code>expertProvider</code> , <code>skillsProvider</code> , <code>instantWorkProvider</code> , <code>networkStatusProvider</code>	Mutable screen state with <code>copyWith</code> : <code>loading/data/error/offline/cachedTimestamp</code>
<code>AsyncNotifier</code> / <code>FutureProvider</code> / <code>StreamProvider</code> / <code>ChangeNotifierProvider.family</code>	<code>notificationsProvider</code> , <code>eventsProvider</code> , <code>myApplicationsProvider</code> , <code>myInterviewsProvider</code> , <code>citiesFutureProvider</code> , <code>conversationsProvider</code> , <code>websocketStatusStreamProvider</code> , <code>chatMessagesProvider.family(conversationId)</code>	One-shot fetches, streams, per-conversation chat state

Key providers — behaviour summary

Provider	Behaviour
authProvider (Notifier)	Boot: restore cached user → refresh profile; auto-refresh on reconnect; all auth/profile actions set <code>isLoading/error</code> ; <code>_parseError()</code> produces friendly messages; offline never logs you out; optimistic project add on API failure.
jobsProvider (Notifier)	Loads cache instantly → fetches network data; stale-response race guard (<code>_currentRequestId</code>); filter setters reset page to 1 and refetch; bookmarks persisted locally; auto-refetch on reconnect; error → cache-with-timestamp, else offline/error state.
walletProvider (Notifier)	Money safety: requires online + single-flight <code>isActionLoading</code> lock for add/withdraw/transfer; success → parallel refresh of profile + transactions; rethrows <code>WalletApiException</code> for backend-pending messages.
notificationsProvider (AsyncNotifier)	Skips network when logged out; <code>markAsRead</code> / <code>markAllAsRead</code> are local-only; unread badge derived, not hardcoded.
eventsProvider (AsyncNotifier)	Optimistic registration: marks registered + increments count first, rolls back on failure.
skillsProvider / expertProvider	Parallel <code>Future.wait</code> for categories+skills; error messages switch on exception type: <code>AppNotFoundException</code> → "...currently under development", <code>AppNetworkException</code> → "No internet connection..."
instantWorkProvider	Reuses JobRepository with filter <code>jobTypes: ['Instant', 'Hourly', 'Gig', 'Part-time']</code> , <code>limit:30</code> — instant gigs are just filtered jobs.
network providers	<code>pendingInvitationsProvider</code> , <code>connectionsProvider</code> , <code>connectionStatusProvider.family(userId)</code> — pure FutureProviders, invalidated after each action.
chat providers	<code>conversationsProvider</code> (list), <code>websocketStatusStreamProvider</code> (live socket state), <code>chatMessagesProvider.family</code> — per-conversation ChangeNotifier that connects the socket, loads history, appends WS messages, invalidates conversations after send.
networkStatusProvider	Bridges the ConnectivityService stream to Riverpod; <code>refresh()</code> runs the active DNS probe.

9. The API Layer — In-Depth Breakdown

9.1 ApiClient — the single network gateway

```
apiClientProvider = Provider<ApiClient>((ref) => ApiClient());

BaseOptions: baseUrl=https://api.kaammilega.com/api · connectTimeout=15s · receiveTimeout=15s
              headers: Content-Type/Accept = application/json
Public API:  get() · post() · patch() · delete() → all funnel into _safeExecute()
```

Interceptors (one InterceptorsWrapper, three hooks)

Hook	Action	Effect
onRequest	Reads <code>LocalStorage.getToken()</code> → sets <code>Authorization: Bearer <jwt></code>	Auth is automatic in every repository — never attached manually
onResponse	Any response ⇒ <code>ConnectivityService.updateStatus(online)</code>	Server answered ⇒ banner hides itself
onError	401 ⇒ <code>LocalStorage.clearSession()</code> (token+user wiped); connectionError/connectionTimeout ⇒ <code>updateStatus(offline)</code>	Expired session self-heals to guest; network failure raises global banner

`_safeExecute()` logs one structured line (`[ApiClient] HTTP 404 | GET /api/skills - Backend endpoint unavailable`) then throws the mapped `AppException` . The constructor accepts injected `baseUrl/Dio` for tests.

9.2 Error translation & the sealed AppException hierarchy

Type	Trigger	Code	User sees
<code>AppNetworkException</code>	<code>SocketException</code> , <code>connectionError</code> , <code>badCertificate</code> , <code>unknown+socket</code>	<code>NETWORK_ERROR</code>	Offline banner / retry view
<code>AppTimeoutException</code>	send/receive/connect timeout	<code>TIMEOUT</code>	"Connection timed out..."
<code>AppAuthException</code>	HTTP 401/403	<code>AUTH_ERROR</code>	Server message or "Session expired..."
<code>AppNotFoundException</code>	HTTP 404	<code>NOT_FOUND</code>	"This service is currently unavailable" (≠ empty list)
<code>AppServerException</code>	HTTP ≥500	<code>SERVER_ERROR</code>	"Server temporarily unavailable..."
<code>AppValidationException</code>	400/422, cancel	<code>VALIDATION_ERROR</code>	Server message + <code>validationErrors</code> map

Server messages are extracted from body keys `error` → `message` → friendly generic fallback. Because the class is **sealed**, callers can exhaustive-switch. Connection failures also flip connectivity to offline.

9.3 ApiConstants & endpoint catalog

```
baseUrl = 'https://api.kaammilega.com/api' // already includes /api – never prepend it again

Auth      : /auth/otp/send|verify · /auth/login/password · /auth/register/password
           /auth/password/forgot|reset · /auth/otp/email/send|verify
Profile   : /user/profile · /user/register · /user/skill · /user/education · /user/experience · /user/project
Jobs      : /jobs · /jobs/{id}      Applications : /applications · /applications/my
Interview : /interviews/my          Cities       : /cities      Skills : /skills
Network   : /network/connect|accept|ignore|pending|connections|connections/{id}|status/{id}
Chat      : /chats · /chats/messages · /chats/{id}/messages · WS /ws/chats
Feed      : /posts · /feed          Notifications : /notifications
Events    : /events · /events/{id}/register      Mentorship : /mentorships · /mentorships/{id} · /mentorships/book
Files     : /files/upload                Admin : /admin/companies      Wallet : /wallet/* (in WalletRepository)
```

`resolveImageUrl()` normalises image refs: strips stray `file:///api/...` prefixes, passes through `http(s)://`, `assets/`, `data:`, `blob:`, and joins relative paths to the host with the trailing `/api` removed (`/files/x.png` → `https://api.kaammilega.com/files/x.png`). Used by notification avatars, expert images, event images, profile photos.

9.4 Repository layer — all 12 repositories

① AuthRepository (auth, profile, files)

Method	HTTP	Endpoint	Payload / behaviour
sendOtp / verifyOtp	POST	/auth/otp/send · verify	{mobile, code, role:'user'} → saves token+user
loginWithPassword	POST	/auth/login/password	{email, password, role} → saves token+user
registerWithPassword	POST	/auth/register/password	{name, email, password, role}
forgotPassword / resetPassword	POST	/auth/password/forgot · reset	{email, role} / {email, code, new_password, role}
sendEmailOtp / verifyEmailOtp	POST	/auth/otp/email/*	returns bool (errors → false)
registerCandidate	POST	/user/register	roles[], name, gender, education_level, work_experience, city, job_categories[], ...
updateProfile	PATCH	/user/profile	partial update; response replaces cached user
addEducation/Experience/Skill/Project	POST	/user/education experience skill project	each returns FULL updated user → re-cached
uploadFile	POST	/files/upload	multipart FormData; reads url/file_url; absolutises relative URLs
getProfile	GET	/user/profile	on error returns cached user (offline resume)
logout	—	—	local session clear only

② JobRepository

- `getJobs(JobFilter)` → `GET /jobs?` + `toQueryParams()`; accepts **3 shapes**: `{jobs,total,page,limit}`, bare array, or empty → always a `JobsResponse`.
- `getJobById(id)` → `GET /jobs/{id}`; unwraps `{job:{...}}` or flat; throws if neither.

③ ApplicationRepository — write safety + offline cache

- `applyToJob()` → `POST /applications` {job_id, cover_letter}, but **throws before the request** if offline.
- `getMyApplications()` → `GET /applications/my` (array or {data}); success → writes slim cache; failure → cache or [].

④ ChatRepository

- `GET /chats` list · `POST /chats/messages` {receiver_id, content} · `GET /chats/{id}/messages?limit&offset`; read errors → [].

⑤ NetworkRepository

`POST /network/connect|accept|ignore` ({receiver_id}/{sender_id}) · `GET /network/pending` · `GET /network/connections` ({connections}) · `GET /network/status/{id}` · `DELETE /network/connections/{id}`

⑥ WalletRepository — payments (backend partially pending)

- `GET /api/wallet/transactions` · `POST /api/wallet/add-money` {amount, payment_method} · `POST /api/wallet/withdraw` {amount, destination_type, destination_detail} · `POST /api/wallet/transfer` {amount, recipient, note?}
- Write methods aim to detect 404/501/405 → `WalletApiException(isBackendPending:true)` with explicit dev-facing messages ("Payment Gateway API is not yet mounted on the backend..."). See observation §13.2 — the `on DioException` catch may be unreachable.

⑦ InterviewRepository · ⑧ CityRepository · ⑨ EventRepository

- `GET /interviews/my` (errors → []) · `GET /cities?active=true&limit=100[&search]` with a **hard-coded 9-city offline fallback** (Mumbai... Jaipur) so dropdowns never break · `GET /events?page&limit&search&location` ({data})[{events}]array + `POST /events/{id}/register` → 2xx success.

⑩ SkillsRepository · ⑪ ExpertRepository · ⑫ NotificationRepository

- `GET /skills?q&category` + `GET /skills/categories` · `GET /mentorships?category` + `GET /mentorships/{id}` + `POST /mentorships/book` {mentorship_id, scheduled_at, notes?} (free-text search done client-side) · `GET /notifications` ({notifications})array — **the only repo that lets AppExceptions propagate**, which powers the "under development" screen.

9.5 WebSocket real-time chat

```
URL build: baseUrl https://api.kaammilega.com/api
           scheme 'https' → 'wss' (else 'ws')
           wss://api.kaammilega.com/api/ws/chats?token=<JWT> (8 s handshake timeout)

Guards: disposed? · already connected? · offline? · no token? → bail out
Inbound: String frame → jsonDecode → type=='NEW_MESSAGE' → ChatMessage.fromJson
         → dedup via _processedMessageIds Set → broadcast Stream
         → ChatMessagesNotifier filters conversationId → id-dedup again → notifyListeners() → UI
Reconnect: up to 5 attempts, delay = attempt × 2s (2,4,6,8,10); triggers on error/onDone/failed connect
           + auto-reset & connect the instant ConnectivityService reports online
Status:   disconnected → connecting → connected | error (WebSocketStatusStreamProvider)
Dispose:  cancels network sub, timer, socket, closes both controllers (ref.onDispose)
```

open /chats/id → ws.connect() → GET history → POST send → WS live append → invalidate conversations

History via REST, delivery via WS, merged in one list with id-based dedup — messages never duplicate.

9.6 Session lifecycle & endpoint status matrix

#	Step	Detail
1	Cold start	<code>LocalStorage.init()</code> before first frame
2	Session check	token present → instantly authenticated + cached user hydrates the UI
3	Server sync	<code>GET /user/profile</code> ; failure ignored while offline
4	Every call	interceptor injects <code>Bearer</code>
5	Token expires	401 → interceptor wipes session → guest state → re-auth (OTP/password)
6	Logout	local clear only (no server revoke) + <code>AuthState</code> reset
7	Realtime	same JWT passed as <code>?token=</code> on WS handshake

Feature	Method	Endpoint	Backend	Fallback behaviour
OTP / password auth	POST	<code>/auth/*</code>	Active	snackbar / session saved
Profile get/update, skills, education, experience	GET/PATCH/POST	<code>/user/*</code>	Active	cached offline
Jobs list/detail · applications · interviews · cities	GET/POST	<code>/jobs</code> , <code>/applications</code> , <code>/interviews/my</code> , <code>/cities</code>	Active	cached + timestamp badge
Network actions · chats · WebSocket	GET/POST/WS	<code>/network/*</code> , <code>/chats</code> , <code>/ws/chats</code>	Active	snackbar / live messaging
Notifications	GET	<code>/notifications</code>	Pending/404	"under development" + retry
Skills catalog/categories	GET	<code>/skills</code> , <code>/skills/categories</code>	Pending/404	"Skills catalog unavailable" + retry
User peer search	GET	<code>/users/search</code>	Pending/404	"User discovery unavailable"
Events · mentorships	GET	<code>/events</code> , <code>/mentorships</code>	Pending/404	clean retry view
Wallet add/withdraw/transfer	POST	<code>/wallet/*</code>	Pending	<code>WalletApiException</code> backend-pending message

9.7 End-to-end request traces

Trace A — Apply to a job

Tap Apply → AuthPrompt if guest → ApplyModalSheet (cover letter) → online check → Bearer injected → POST /api/applications
→ success → invalidate myApplications → GET /applications/my → cache

- Button disabled while submitting (double-tap guard); offline ⇒ `AppNetworkException` before any request.
- Failure path: `DioException` → `onError` (401 clears session / connection sets offline) → `_handleError` → `AppException` → Notifier catches → `NetworkStateView` renders offline/error/retry.

Trace B — Search for jobs



Trace C — Receive a chat message while offline→online



10. Feature-by-Feature Screen Walkthrough (all 45 screens)

10.1 Auth module

Screen	UI	Logic & navigation
LoginScreen (650 ln)	Segmented OTP ↔ Password tab switcher, phone/email + password forms, footer links, brand logo	<code>sendOtp(phone)</code> → push /otp with phone extra; <code>loginWithPassword</code> → go /home; errors from authProvider; push /register, /forgot-password; guest shortcut → /jobs
OtpScreen (184 ln)	4/6-digit code entry, resend timer	<code>verifyOtp</code> → registered? go /home : go /register; resend re-calls <code>sendOtp</code>
RegisterScreen (499 ln)	Name/email/password + confirm, terms checkbox, matches web design	<code>registerWithPassword</code> → success go /home; errors surfaced inline; link → /login
ForgotPasswordScreen (~800 ln)	2-step wizard: Step 1 email → send 4-digit code; Step 2 code + new password + confirm	<code>forgotPassword(email)</code> then <code>resetPassword(email, code, newPassword)</code> → go /login; both steps have their own error handling

10.2 Home, navigation & discovery

Screen	UI sections	Data / actions
HomeScreen (1,588 ln)	Top header (city selector, search, bell w/ unread badge, drawer), InstantMilega banner, quick-action icon grid, popular categories, "Recommended for you" horizontal job cards + shimmer, Top Picks, ₹99 Access banner, benefits rows	watches <code>jobsProvider</code> + <code>unreadNotificationsCountProvider</code> ; city change → <code>jobsProvider.setCity</code> ; search → <code>setSearchQuery</code> then go /jobs; quick actions push /instant-work, /skills-marketplace, /experts, /services, /explore; job tap → push /jobs/:id (with Job extra); apply checks auth
MainNavigationShell	5-tab IndexedStack + custom bottom bar + center (+) Quick Actions sheet	see \$4
ProfileDrawer	Unified navigation drawer used across Home/Jobs/Chats/Apply-Expert/Profile: header with avatar + unread badge, primary links, full menu	routes to /home /jobs /notifications /profile/login /network /events /experts /chats /feed /explore /wallet /settings /my-applications /interviews /apply-expert; logout → confirm → go /home
ExploreScreen	Category hub grid of module cards (themed colours) routing to all marketplaces	<code>context.push(route)</code> per card; plain StatefulWidget (no API)
CitySelectorSheet	Modal bottom sheet with search + city list (mirrors website header selector)	watches <code>citiesFutureProvider</code> ; selection persisted via LocalStorage + applied to jobs filter
FeedScreen	Clean "Resources" empty-state container (no social clutter yet)	back → /home; future: /posts feed

10.3 Jobs & applications

Screen	UI	Logic
JobsScreen (920 ln)	Hero header w/ metrics, search bar (400 ms debounce), city selector, filter modal trigger + active-filter count, promo & Top-Match banners, <code>JobCard</code> list w/ bookmark toggle, pagination bar, saved-jobs link, pull-to-refresh	watches <code>jobsProvider</code> , <code>myApplicationsProvider</code> , unread badge; apply requires auth; call HR via <code>url_launcher (tel:)</code> ; chat → <code>/chats</code> ; page change → <code>setPage</code> ; clear-all filters
FilterModalSheet	Accordion filters: job type, salary, experience, gender, qualification (mirrors website)	builds a new <code>JobFilter</code> → <code>jobsProvider.applyFilter</code> (page resets to 1)
JobDetailScreen (1,471 ln)	Company header, salary/experience/location chips, description, requirements & "We offer" lists, vacancies/applicant stats, sticky Apply/Save/Call/Chat actions	loads job by id (falls back to cached list / <code>extra</code> object); open apply modal; already-applied state from <code>myApplications</code> ; auth-gated actions
ApplyModalSheet	Bottom sheet: optional cover letter, submit button	auth check → <code>applyToJob</code> → push <code>/applications/:id</code> on success
MyApplicationsScreen	Application cards with status pills (Applied/Shortlisted/Interview/Rejected), interviews shortcut, empty state → <code>/jobs</code>	<code>myApplicationsProvider</code> + pull-to-refresh; tap → <code>/applications/:id</code>
ApplicationDetailScreen	Status timeline/detail matching the web UI	derives item from <code>myApplications</code> list; job link → push <code>/jobs/:id</code> ; missing → go <code>/jobs</code>
SavedJobsScreen	Bookmarked list with search + remove; empty state	reads <code>jobsProvider.savedJobIds</code> ; tap → job detail; onChat → <code>/chat</code>
InterviewsScreen	Interview cards: date/time ("EEE, d MMM yyyy • h:mm a"), mode, location, recruiter notes	<code>myInterviewsProvider</code> ; empty → go <code>/jobs</code>

10.4 Chat, network & notifications

Screen	UI	Logic
ChatListScreen (585 ln)	Conversation tiles (avatar, name via other-participant, last message, relative time), unread badge in header, "more options" bottom sheet, new-chat action	<code>conversationsProvider</code> + <code>currentUser</code> + unread count; pull-to-refresh; tap → push <code>/chats/:id</code> with <code>{receiverId,title}</code>
ChatDetailScreen (338 ln)	Bubbles (mine vs theirs), timestamps, auto-scroll to bottom, composer with send button, live connection indicator from WS status	<code>chatMessagesProvider.family(id)</code> → connects socket, loads history, appends WS messages; <code>sendMessage</code> POST + invalidate conversations
NetworkScreen	Two tabs: Active Connections & Pending Invitations; per-item Accept / Ignore / Delete / Chat	watches <code>connectionsProvider</code> + <code>pendingInvitationsProvider</code> ; actions call <code>networkRepository</code> then invalidate providers; chat → <code>/chats</code>
PeerToPeerScreen	Peer discovery + invite/accept UI (module variant of networking)	reads <code>apiClientProvider</code> directly for discovery, uses <code>networkRepository</code> for send/accept; shows pending + connections; push to chat
NotificationsScreen	Notification rows (avatar, title, message, relative time, unread highlight), "Mark all read", empty/offline/login states	auth-gated <code>notificationsProvider</code> ; tap → <code>markAsRead</code> + <code>push(item.targetRoute)</code> ; refresh button

10.5 Wallet (5 screens)

Screen	UI	Logic
WalletScreen (1,020 ln)	Deep-navy balance card + verification badge, 4 quick actions (Add/Withdraw/Transfer/History), wallet-types breakdown (Main/Earnings/Locked/Bonus), KYC & limits card, recent transactions with zero-mock empty state, info & "backend required" modals	watches isOnline + auth + walletProvider; pull-to-refresh → refreshWallet(); actions push /wallet/add-money withdraw transfer transactions
WalletAddMoneyScreen	Amount presets + custom amount, payment method choice	requires online (isOnlineProvider else AppNetworkException) → walletProvider.addMoney → single-flight lock → back to /wallet
WalletWithdrawScreen	Amount + destination (bank/UPI) fields, fee hint	online check → withdraw() with destination_type/detail
WalletTransferScreen	Recipient identifier + amount + note	online check → transfer()
WalletTransactionsScreen	Ledger list w/ credit-debit filter, status pills (Completed/Pending/Failed), pull-to-refresh	watches walletProvider; back → /wallet

10.6 Explore modules

Screen	UI	Data
InstantWorkScreen	Gig feed with filter chips (All/Today/Hourly/Urgent/Nearby), search, shimmer, empty → /jobs	instantWorkProvider — JobRepository filtered to Instant/Hourly/Gig/Part-time (limit 30)
SkillsMarketplaceScreen	Category pills + search over skill catalog, user's own skills section	skillsProvider (categories+skills in parallel); auth-aware links to /profile and /jobs
ExpertsScreen	Mentorship cards: expert avatar/headline, category, rating, price, duration; "Become an Expert" CTA	expertProvider ; 404 → "under development"; push /apply-expert
ExpertDetailScreen	Expert profile + session details + book action	reads expertRepositoryProvider directly; auth-gated booking → POST /mentorships/book
ApplyExpertScreen	Application form (bio, pricing, category, proof docs)	pushes to /jobs on completion; shows unread badge
ServicesMarketplaceScreen	Service categories (electrician, plumber, technician, carpenter) & provider cards	mostly static scaffold; CTA → /apply-expert
EventsScreen	Event cards w/ category streams, shimmer, pull-to-refresh, register button	eventsProvider ; auth-gated register (optimistic); push /login when guest
EventDetailScreen	Event info + register CTA	auth check → register; guest → push /login
CompanyScreen	Company banner/logo + jobs-at-company list	companyId path param; follow/apply auth-gated

10.7 Profile & settings

Screen	UI	Logic
ProfileScreen (6,781 ln — largest file)	Full CV: cover + avatar (upload/delete modals), headline/about editors, profile-strength meter, skills tags, education list, experience list, projects, portfolio links, resume, wallet shortcut, connections shortcut, stats (views/impressions), logout	~15 auth-gated modal flows (edit profile, about, portfolio, add/edit education/experience/project, photo/cover/resume); photo/cover/resume = uploadFile → updateProfile ; pull-to-refresh → refreshProfile; push /wallet, /network, /login, /jobs
SettingsScreen	Notification toggles, privacy/profile visibility mode, change password (current/new/confirm), preferences & language	watches authProvider.user; local toggles + password change via authProvider

10.8 Splash & system

SplashScreen: LocalStorage.init → 1.5 s timer → go /home (token or guest — same destination) · **OfflineBannerOverlay:** global (see §11) · **NetworkStateView:** global state machine (see §6).

11. Offline Resilience & Caching System

(1) ConnectivityService — singleton source of truth

- Wraps `connectivity_plus`; exposes `currentStatus`, `isOnline`, broadcast `statusStream`.
- `updateStatus()` lets the API layer override the OS signal (any response = online; connection failure = offline).
- `probeInternet()` — on-demand active check (no polling): DNS lookup of `kaammilega.com`, fallback `google.com`, 3 s each.
- Test factory `ConnectivityService.forTest()` for unit tests.

(2) Riverpod bridge

`networkStatusProvider` (Notifier) mirrors the stream into widget state; `isOnlineProvider` is the boolean read by wallet/apply guards; banner "Check" button → `refresh()` → active probe.

(3) OfflineBannerOverlay — global UX

Installed in `MaterialApp.router.builder` so it covers every screen. Animated slide-down "No Internet Connection / Some features may be unavailable. Showing saved data." (dark slate, red border, wifi-off icon, **Check** button) and a green **"Back Online"** banner for 2.5 s on recovery.

(4) NetworkStateView — standard 5 states

State	Condition	UI
Loading	<code>isLoading</code>	Custom loader or shimmer skeleton
Offline	<code>isOffline</code>	wifi-off icon + message + Retry
Error	<code>errorMessage</code>	error icon + message + Retry
Empty	<code>isEmpty</code>	"No Records Found" (only for genuine 200-[])
Cached	<code>cachedTimestamp != null</code>	Amber badge "Showing saved data (Updated: ...)" above real content

Cache inventory (SharedPreferences)

Key	Content	Writer
<code>km_auth_token</code>	JWT	AuthRepository
<code>km_user_data</code> + <code>_time</code>	Profile JSON + timestamp	every profile mutation
<code>km_cache_jobs</code> + <code>_time</code>	Job list snapshot	JobsNotifier
<code>km_cache_applications</code> + <code>_time</code>	My applications snapshot	ApplicationRepository
<code>km_cache_wallet</code> + <code>_time</code>	Transactions snapshot	WalletNotifier
<code>km_saved_job_ids</code> · <code>km_selected_city</code>	Bookmarks · preferred city	bookmark toggle / city sheet
generic <code><key></code> + <code><key>_time</code>	<code>setGenericCache/getGenericCache</code> helpers	any caller

The recovery loop

request fails → `ConnectivityService=offline` → banner shows → repositories serve cached snapshots with timestamp → connectivity flips online → every Notifier's `ref.listen` fires → automatic refetch → banner hides, badge clears.

12. Testing, Git History & Existing Documentation

Test suite	What it verifies
<code>network_resilience_test.dart</code>	Network layer: NetworkStatus extension booleans · ConnectivityService status updates emit on stream · default messages of AppNetworkException/AppTimeoutException/AppAuthException · ApiClient converts a rejected <code>DioExceptionType.connectionError</code> into <code>AppNetworkException</code> (mocked SharedPreferences + injected Dio)
<code>wallet_test.dart</code>	WalletTransaction fromJson/toJson round-trip incl. enums (credit/debit, completed) and reference_id · widget tests rendering WalletScreen, WalletTransactionsScreen, WalletAddMoneyScreen, WalletWithdrawScreen with a seeded auth user
<code>explore_modules_test.dart</code>	ExpertItem dual-format parsing (nested <code>{mentorship, expert}</code> AND flat JSON) · SkillItem parsing · ExploreScreen / ServicesMarketplaceScreen render · CategoryTopHeader & ThemedCategoryBottomNav widgets
<code>widget_test.dart</code>	Job model fromJson + formattedSalary ("₹20,000 - ₹30,000") + formattedExperience ("1-3 Yrs") · JobFilter/JobsResponse · SavedJobsScreen & ApplyExpertScreen & ProfileDrawer widgets

Git history (branch `main`, 249 tracked files)

Recent evolution: Phase 1–4 core implementation → shimmer everywhere → website-aligned UI + onboarding → candidate profile/auth/settings/application tracking → feed, notifications, events, bookmarking → profile drawer + logo → login OTP/password + status tracking → jobs/chats tab revamp + Apply Expert → wallet module → explore modules + themed nav + mentorship fix → profile modals revamp → **latest: offline resilience foundation + clean 404 API handling.**

Existing docs in repo: `API_INTEGRATION_STATUS.md` (endpoint matrix + client rules) · `flutter_app_feature_tracker.md/.csv` (72 features:  35 /  17 /  5 /  15) · `KAAMMILEGA_FLUTTER_PROJECT_DOCUMENTATION.md/.pdf` · `README.md` (stock Flutter readme).

13. Observations, Risks & Recommendations

Analysis only — no code was changed, as requested.

13.1 Wallet paths appear double-prefixed with /api

`baseUrl` already ends in `/api`, and the project's own guidance says never prepend `/api` — but `WalletRepository` calls `get('/api/wallet/transactions')`, `post('/api/wallet/add-money')`, etc. Dio concatenates `baseUrl+path`, so requests may hit `.../api/api/wallet/...` → 404 silently converted to an empty list. **Verify with a network log**; the fix (when you choose to make it) is to drop the leading `/api` there.

13.2 on DioException catches in WalletRepository are likely unreachable

`_safeExecute()` converts every Dio failure into an `AppException` **before** it leaves `ApiClient`, so the wallet's `on DioException ... 404/501/405 → isBackendPending` blocks fall through to `catch (_)`. The friendly "gateway not yet mounted" messages are therefore hard to trigger. Catching `AppNotFoundException` / `AppServerException` instead would restore the intent.

13.3 Several repositories swallow errors with catch (_) → []

Chat, Network, Interview (and wallet reads) collapse every failure — including 404 — into an empty list, which conflicts with the documented rule "404 = service unavailable, never an empty list". Notifications does it correctly (it propagates), which is why only that screen shows the under-development card. Skills/Experts also propagate — good.

13.4 Connectivity "override" can latch

Both interceptors call `updateStatus()` on every response/error, which sets `_isOverridden = true` and makes the OS `connectivity_plus` stream ignored until `probeInternet()` / `clearOverride()` runs. Behaviour is still sensible (the API is the better signal) but worth knowing when debugging the banner.

13.5 Security & release readiness

- JWT stored in plain **SharedPreferences** (readable on rooted devices; included in backups) → consider `flutter_secure_storage` (Keystore/Keychain).
- No token refresh / no server-side logout — 401 silently drops the user to guest mid-session.
- Android `applicationId` is still `com.example.kaam_milega` and release builds are **signed with debug keys** — both must change before store submission.
- WhatsApp/call HR uses `url_launcher` — ensure `tel:` / `https` intent queries are declared for Android 11+.

13.6 Code-health observations

- **profile_screen.dart is 6,781 lines** and `home/job_detail` pass 1,400 — prime candidates to split into feature widgets/panels.
- Docs mention `PUT /user/profile` but code uses `PATCH` (`ApiClient` has no `put()`).
- Some endpoints are hard-coded in repositories (`/user/education`, `/files/upload`, `/wallet/*`) instead of living in `ApiConstants`.
- Center (+) "Instant Work Request" shows a success snackbar but performs no API call.
- Chat tab unread badge is hard-wired to `0` ("update when backend supports it").
- `markAsRead` for notifications is local-only (no write-back endpoint yet).
- Free-text expert search filters client-side after fetching the whole list.
- Splash always routes to `/home` regardless of token (guest-first design) — fine, but the token check is dead code.

13.7 What is genuinely well built

- One hardened network gateway: automatic Bearer injection, 15 s timeouts, structured single-line logs.
- Sealed `AppException` hierarchy = exhaustive typed error handling across the app.
- Coherent offline story: global banner + per-screen 5-state view + caches with timestamps + auto refetch on reconnect.
- Race-guard in JobsNotifier stops stale responses overwriting fresh data; 400 ms search debounce.
- Money flows are connectivity-checked with a single-flight duplicate-submission lock.
- WebSocket: token handshake, dedup, bounded backoff, connectivity-aware recovery, clean disposal.
- Optimistic UI done right in events (with rollback) and profile projects (local fallback).
- Honest "endpoint not ready" UX instead of fake data — and unit tests covering exactly the risky parts (error translation, model parsing, wallet round-trips).
- Consistent design system (colors/typography/branding as single sources of truth) and a genuinely reusable widget kit.